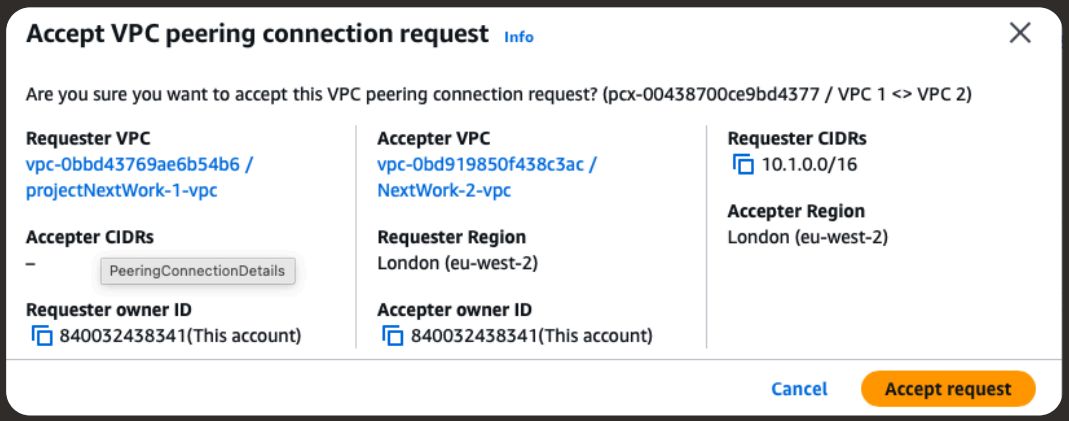
VPC PEERING

By Nabil Ahmed | September 2026

VPC Peering



Introducing Today's Project!

What is Amazon VPC?

Amazon VPC is AWS's foundational networking service that lets us create our own networks, control traffic flow and security and it is useful because it can help us organise our resources into public and private subnets.

How I used Amazon VPC in this project

In today's project, I used Amazon VPC to set up a multi VPC architecture and to create a peering connection and update security rules to run a successful connectivity test to validate my VPC peering set up.

One thing I didn't expect in this project was...

One thing I didn't expect in this project was needing a public IPv4 address for EC2 Instance Connect to work. Also I did not expect that Elastic Ips can assign static public IPv4 addresses.

This project took me...

This project took me 1 hour and 30 minutes to complete including time to document what I have learnt.

In the first part of my project...

Step 1 - Set up my VPC

In this step, I will be using the VPC map/launch wizard to create two VPCs and their components because this is the quickest way to set up VPCs as I won't have to set up things like route tables and internet gateways multiple times for two VPCs.

Step 2 - Create a Peering Connection

In this step, I will set up a VPC Peering Connection, which is a VPC component designed to directly connect two VPCs together.

Step 3 - Update Route Tables

In this step, I will ubdate the route tables for each of the VPCs because currently there is no route that enables the traffic from one VPC to go to the Peering Connection.

Step 4 - Launch EC2 Instances

In this step, I will launcg an EC2 instance in each of the VPCs so that I can directly connect with my instances later and test the VPC peering connection.

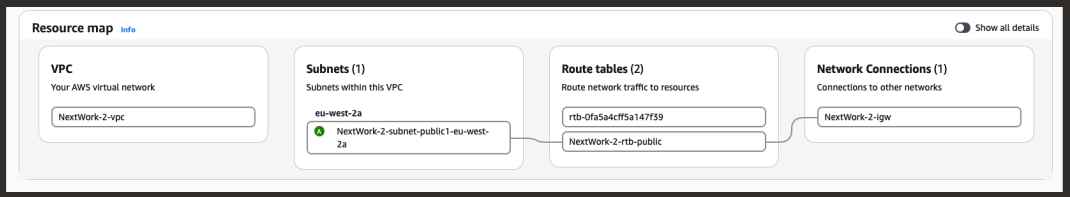
Multi-VPC Architecture

I started my project by launching creating two VPCs - they had unique cidr blocks and each had one public subnet.

The CIDR blocks for VPCs 1 and 2 are 10.1.0.0/16 and 10.2.0.0/16. They have to be unique because once you set up a VPC peering connection, route tables need unique addresses for correct routing across VPCs.

I also launched 2 EC2 instances

I didn't set up key pairs for these EC2 instances as I am using EC2 Instance Connect to directly connect with my EC2 instances later in this project, which handles key pair creation and management for me.

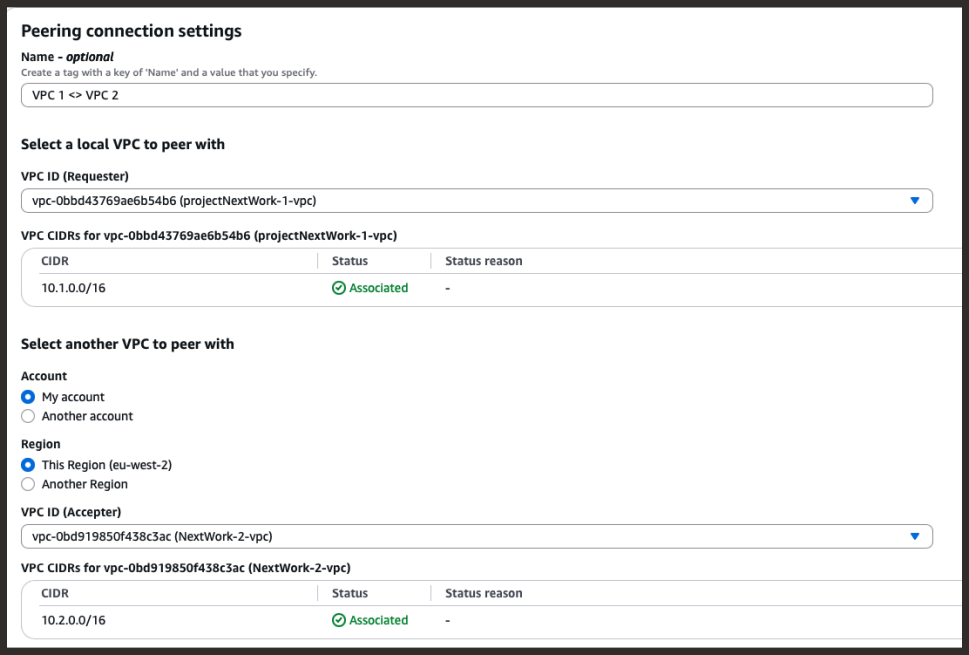


VPC Peering

A VPC peering connection is is a networking link between two VPCs allowing them to exchange traffic privately, protecting them from having to send data through the public internet where they could potentially be intercepted.

VPCs would use peering connections to exchange traffic securely between two VPCs without having to go through the public internet.

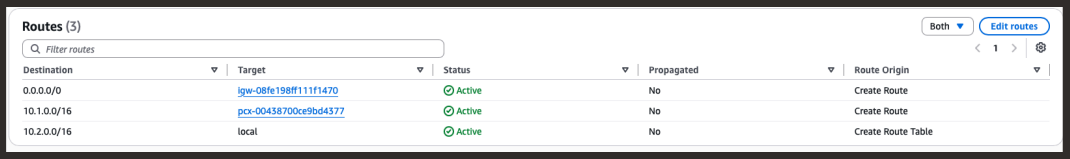
The difference between a Requester and an Accepter in a peering connection is a Requester is the side which initiates a connection request. An Accepter is the side that recieves and must approve the request to activate the connection.



Updating route tables

After accepting a peering connection, my VPCs' route tables need to be updated because the default route table doesn't have a route using the peering connection yet - this needs to be set up so resources can be directed to the peering connection when trying to reach the other VPC.

My VPCs' new routes have a destination of the other VPC's cidr block. The routes' target was the peering connection.



In the second part of my project...

Step 5 - Use EC2 Instance Connect

In this step, I will use EC2 Instance Connect to connect directly with my first EC2 instance. I need to do this because I need to use my EC2 instance for connectivity tests later in this project.

Step 6 - Connect to EC2 Instance 1

In this step, I will resolve another error that is preventing me from using EC2 Instance Connect to directly connect with the instance.

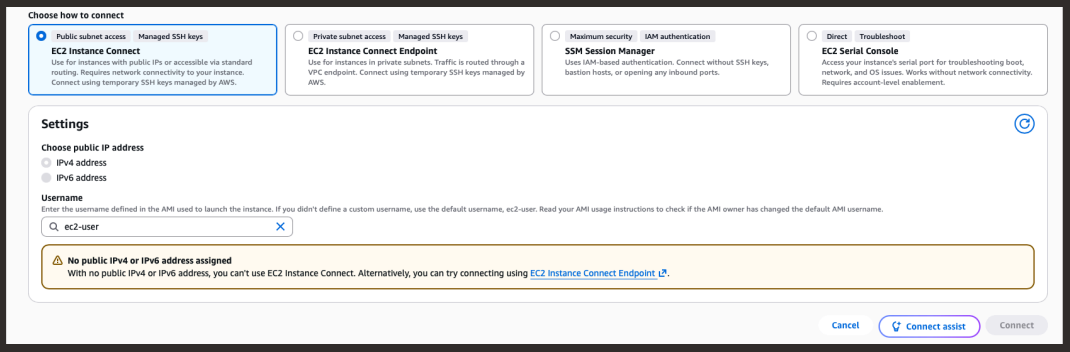
Step 7 - Test VPC Peering

In this step, I am going to use Instance - NextWork VPC 1 to attemot a direct connection with Instance - Nextwork VPC 2 so that I can validate that my peering connection is set up properly.

Troubleshooting Instance Connect

Next, I used EC2 Instance Connect to directly connect with Instance - NextWork VPC 1 just by using the AWS Management Console.

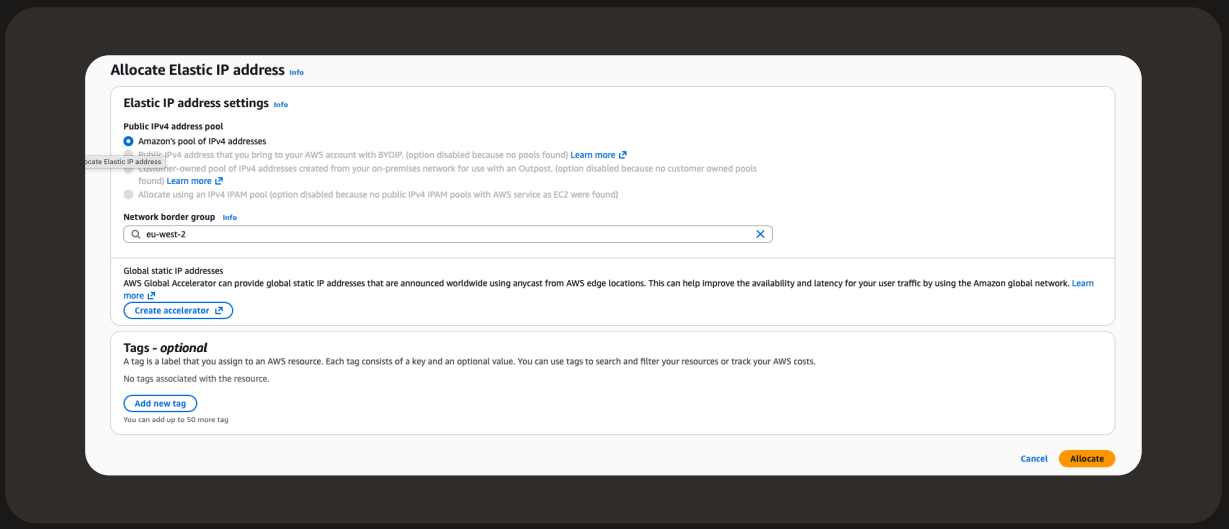
I was stopped from using EC2 Instance Connect as my instance did not have a public IPv4 address. In order for EC2 Instance Connect to work, the EC2 isnatnce must have a public IPv4 address and be in a public subnet.



Elastic IP addresses

To resolve this error, I set up Elastic IP addresses. Elastic IP addresses are static IPv4 addresses that you can request for your AWS account and then delegate to specific resources.

Associating an Elastic IP address resolved the error because it gives my Instance a public IP address. Fullfilling all the requirements for Instance Connect to work.



Troubleshooting ping issues

To test VPC peering, I ran the command ping 10.2.9.47 i.e. the private IPv4 address of the other EC2 isnatce in VPC 2.

A successful ping test would validate my VPC peering connection because this ping test would not get any replies from the other EC2 instance if the peering connection did not successfully connect mt two VPCs.

I had to update my second EC2 instance's security group because it was not letting in ICMP traffic - which is the traffic type of a ping message. I added a new rule that allows ICMP traffic coming from any resource in VPC 2.